ForestGuard

Riccardo Rambaldi
riccardo.rambaldi4@studio.unibo.it

October 21, 2026

ForestGuard is a BDI multi-agent system that simulates a ranger station in the middle of a forest, which monitors the area and fights fires using a swarm of flying drones. The system involves three kinds of agents:

- *Scout drones*, in charge of exploring the forest and reporting what they perceive (trees, water sources and fires) to the station;
- *Firefighter drones*, dispatched by the station to the reported fires, which extinguish them while managing their own battery and water tank;
- the *Ranger Station*, a stationary agent that aggregates the global map of the forest and coordinates the firefighters.

The station can reach any drone through a long-range antenna, while drones can talk to each other only when close: firefighters exploit this short-range communication to negotiate which of them takes care of each burning cell. The agents are written in AgentSpeak and run on the Jason platform, while the forest model, the environment and the graphical simulation (Java Swing) are written in Java. The resulting system is able to autonomously explore a randomly generated forest, detect fires as they ignite and spread, and extinguish them with a mix of centralized dispatching and local coordination, without any human intervention.

1 Goal/s of the project

Forest fires are a growing threat: a single burning tree can quickly spread to the surrounding vegetation, and the longer a fire stays unnoticed, the harder it becomes to contain it. Two factors are therefore crucial: detecting fires as early as possible, and intervening in a coordinated way, so that the available resources are not wasted.

ForestGuard addresses this problem through a simulation in which a forest is guarded by a ranger station and a swarm of autonomous flying drones. The forest is a randomly generated 2D grid containing trees, clearings and lakes, with the ranger station in its center. Fires ignite periodically on random trees and spread to the neighboring ones over time; a tree left burning for too long is lost.

1.1 Goals

The main goal of the project is to realize an autonomous and efficient system of drones that must explore the forest and extinguish fires as fast as possible, while managing limited resources such as water and battery. This goal can be decomposed into four sub-goals:

- **Monitoring**: the whole forest must be continuously explored, so that fires are detected as soon as they ignite;
- **Fire suppression**: detected fires must be reached and put out before they spread;
- **Resource management**: drones have a limited battery and, in the case of firefighters, a limited water tank, which they must manage autonomously without interrupting the service;
- **Coordination**: the station must coordinate the drones from a global point of view, while nearby firefighters must coordinate locally to avoid wasting effort.

The system is considered successful if fires are extinguished before they consume the trees and before they spread out of control, without any human intervention.

1.2 Constraints

The problem is subject to the following constraints:

- **Asymmetric communication**: thanks to its long-range antenna, the station can communicate with any drone at any time, while drones can only communicate with each other when they are close; coordination must therefore be partially centralized and partially local;
- **Partial observability**: each drone only perceives the cells around its position, and the station perceives nothing directly: its knowledge of the forest comes entirely from the drones' reports;
- **Dynamic environment**: fires start and spread independently of the agents, so the knowledge of the agents can quickly become outdated;
- **Limited resources**: every action (moving, extinguishing, refilling) consumes battery or water, so agents must continuously trade off their mission against their resources.

1.3 Example scenario

A typical interaction unfolds as follows:

1. a scout drone, while exploring its area, perceives a burning tree and sends the list of cells it sees to the station;

2. the station updates its global map, notices the new fire and dispatches the closest firefighter waiting at the base;

3. the firefighter claims the burning cell, notifying the nearby firefighters so that they do not target the same cell, and flies towards it;

4. once on the cell, it extinguishes the fire and informs the station, which updates the map and tells the other firefighters that the cell has been handled;

5. the firefighter moves on to the nearest burning cell it perceives, if there is one, until the fire is put out or its tank runs empty;

6. if the tank is empty, the firefighter notifies the station, which answers with the position of the nearest known water source; if the lake is reachable with the remaining battery the firefighter refills there, otherwise it returns to the station to recharge and refill;

7. if instead no other burning cell is in sight, the firefighter goes back to its pad;

8. at any moment, if a drone's battery gets too low to safely reach the station, the drone abandons its current task, releases its target to the other firefighters and goes back to recharge.

2 Analysis

2.1 Functional requirements

The functional requirements are grouped by the goal they contribute to.

Monitoring

1. Each scout explores its own quadrant of the forest with memory, preferring the cells it has visited the least and avoiding stepping back to the cell it just left.

2. At every step, a scout sends to the station the state of all the cells it currently perceives.

3. The station maintains a global map of the forest, updated with the scouts' reports and with the outcome of the firefighters' interventions.

Fire suppression

1. When a scout reports a new burning cell, the station dispatches the closest firefighter waiting at the base; if none is available, it assigns the fire to one of the other firefighters in round-robin.

2. A firefighter reaches its target cell and extinguishes it; it then moves to the nearest burning cell it perceives, until no more burning cells are in sight or its tank is empty.

3. A firefighter also reacts autonomously to the burning cells it perceives, even if they were not assigned to it by the station.

4. A firefighter with nothing to do away from the base notifies the station, which sends it back to its pad.

Resource management

1. Every movement consumes battery; a drone goes back to its pad as soon as its residual battery is barely enough to reach it, plus a safety margin. Firefighters consume more battery than scouts, due to their water tank.

2. After recharging, a scout resumes the exploration from the position where it was before going back to the station.

3. Every extinguishing action consumes part of the water tank; when the tank is empty, the firefighter asks the station for the nearest known lake, and refills there only if the lake is closer than the station and the battery is enough to reach the lake and then the station; otherwise it goes back to the station to both recharge and refill.

4. A drone that goes back to recharge abandons its current task; a firefighter also releases its target cell, making it available to the other firefighters.

Coordination

1. The station can communicate with every drone at any time, while drones can communicate with each other only when they are within short range.

2. Before moving towards a burning cell, a firefighter claims it, notifying the nearby firefighters; two firefighters never target the same cell, and conflicts are solved in favor of the closest one, with ties broken by name.

3. When a cell is extinguished, the station and the nearby firefighters are notified, so that no other firefighter keeps moving towards it.

4. A drone perceives the other drones as obstacles, except when they are landed on their own pad.

Simulation

1. The forest is randomly generated at every run, with the station in its center, lakes and trees.
2. Fires ignite periodically on random trees and spread to the neighboring trees; a tree left burning for too long is lost, while an extinguished tree becomes wet and turns back into a regular tree after a while.
3. The simulation is displayed in a graphical interface, and the user can pause it and change its speed.

2.2 Top-down analysis

- **Multi-agent system**: the domain is naturally made of autonomous entities, each with its own goals and limited resources, which operate concurrently in a shared environment, perceive only part of it and can communicate only under some constraints. A multi-agent system maps these features directly: every drone and the station are agents, and the forest is their environment.
- **BDI architecture**: drones must be both proactive, pursuing long-term goals such as exploring an area or extinguishing a fire, and reactive, responding to new fires, messages and low battery as soon as they occur. The BDI model supports both, and allows priorities among intentions to be expressed explicitly: recharging, for instance, overrides any other activity.
- **Jason**: it implements the BDI model through AgentSpeak, whose Prolog-like rules make the decision criteria of the agents (when to recharge, which cell to target, whether a lake is reachable) declarative and easy to read. Its speech-act-based messages match the asynchronous communication required by the domain, and its Java environment interface allows the simulation and the graphical interface to be implemented separately from the agents.
- **Hybrid coordination**: the asymmetric communication constraint suggests a hybrid approach, in which the station coordinates the drones from a global point of view, while firefighters negotiate locally with the ones within short range.

3 Design

This section presents the design of the system, starting from its overall architecture, then describing how the agents interact and behave, and finally the environment in which they operate.

3.1 Architecture

Besides the agents, the system follows the Model-View-Controller pattern:

- **Agents**: the AgentSpeak sources define the behavior of every agent, together with the custom agent classes that provide their initial beliefs and the internal action used by the scouts to explore;
- **Environment**: the mediator between agents and model, which computes the percepts of each drone, executes their actions and owns the simulation clock;
- **Model**: the grid of the forest, its procedural generation and the dynamics of fires, together with the configuration shared by all the other modules;
- **View and controller**: the graphical interface, which renders the forest and the drones, advances the clock at the chosen speed and allows the user to pause the simulation.

Agents never access the model directly: they can only perceive and act through the environment, which keeps the simulated world consistent and enforces the partial observability of the drones.

3.2 Interactions

Agents interact only through asynchronous messages. All messages ask the receiver to achieve a goal, except for `extinguish_fire`, which adds a new belief to the firefighter: it is up to the firefighter to decide when to act on it. Three main interactions can be identified.

Mapping At every step a scout moves, perceives the cells around it and forwards them to the station (Figure 1). This is a one-way interaction: the station never needs to send messages to the scouts.

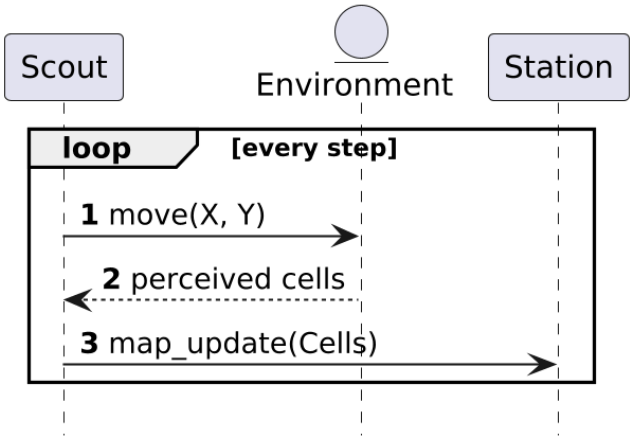


Figure 1: Scout mapping

Detection, dispatch and extinguishing When a scout perceives a burning tree, its position reaches the station together with the other perceived cells. The station then computes the distance between the burning cell and the pads of the firefighters waiting at the base, and dispatches the closest one (Figure 2). The firefighter claims the cell, reaches it, extinguishes the fire and notifies both the station and the nearby firefighters; the station in turn informs all the other firefighters that the cell has been handled.

Since fires usually spread quickly, the dispatched firefighter often perceives other burning cells within its three-cell vision radius: in this case, it keeps track of all of them in a queue and extinguishes them one at a time. When the tank is empty, the firefighter asks the station for the nearest known lake, and the station answers with its position, or sends the firefighter back to its pad if no lake is known.

Finally, firefighters notify the station when they go back to the base and when they land, so that the station always knows which of them are available for a new dispatch.

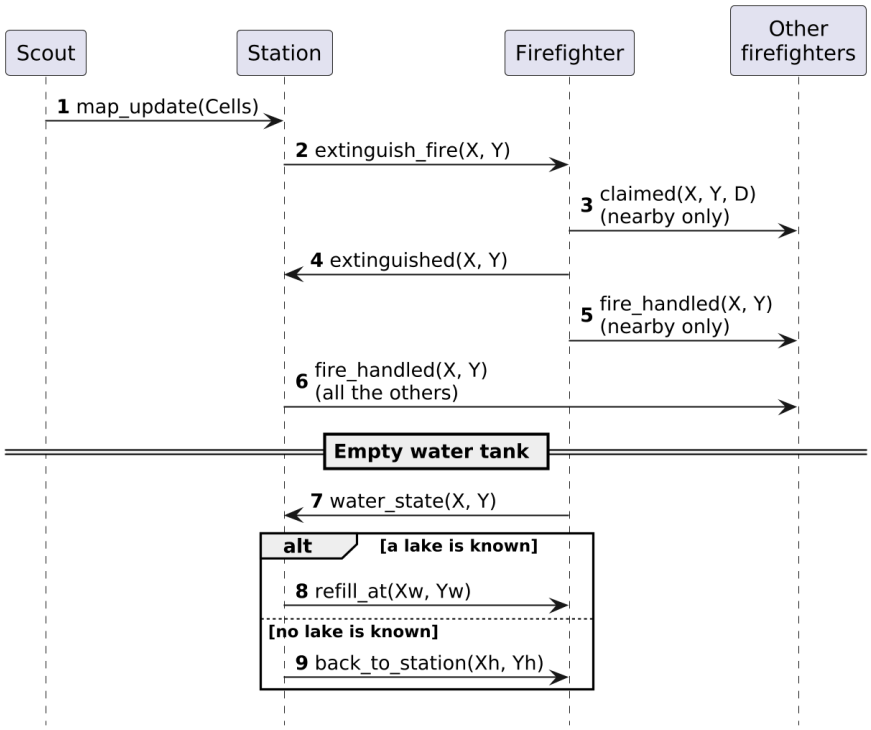


Figure 2: Fire detection, dispatch and extinguishing

Local coordination Since the station dispatches one firefighter for each detected burning cell, and firefighters also react autonomously to the burning cells they perceive, their targets inevitably overlap when several of them work on the same fire. To avoid having multiple drones flying towards the same cell, firefighters coordinate with the ones within short range (Figure 3).

The mechanism works as follows: before moving towards a cell, a firefighter claims it, attaching its distance from the cell; if the receiver is targeting the same cell and is closer (ties are broken by name), it answers with its own claim and the sender gives up; otherwise, the receiver excludes that cell from its targets. A firefighter that abandons its target to refill or recharge releases it, making it available again to the other firefighters, while a firefighter that extinguishes a cell notifies the nearby ones, so that they stop moving towards it.

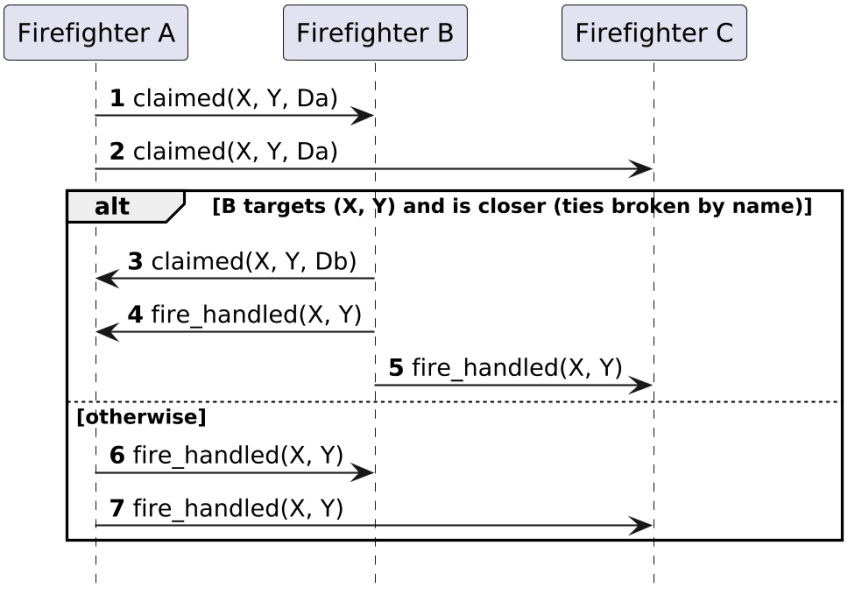


Figure 3: Local coordination between firefighters

3.3 Behavior

All the agents follow the BDI model: their behavior emerges from what they believe about the world (beliefs), the goals they pursue (desires) and the plans they commit to in order to achieve them (intentions). Decisions that depend on the agent's state, such as when to go back to recharge or which cell to target next, are expressed as logical rules over the beliefs, so that each agent always reasons on its most recent knowledge.

Some initial beliefs depend on the configuration of the simulation, such as the size of the map and the position of the pads, and therefore cannot be written statically in the AgentSpeak sources. For this reason, each kind of agent is associated with a custom agent class, which extends the default Jason agent and adds these beliefs when the agent is created, before the AgentSpeak source is loaded. Drones receive the four movement directions, the rule computing the distance between two cells and their battery parameters, while the station receives the position of every pad and the list of firefighters, all initially at the base.

The resulting beliefs are indistinguishable from the ones written in the source files, but they are computed from a single configuration class shared with the environment, so that changing the map does not require any change to the agents.

Scout drone

- **Beliefs**: its position, its battery level, its home pad, the quadrant of the forest assigned to it and how many times it has visited each cell; the cells it perceives are kept only until they are sent to the station.
- **Desires**: exploring its quadrant and keeping the station's map up to date, while never running out of battery.
- **Intentions**: at every step the scout moves towards the least visited free cell around it, avoiding stepping back to the cell it just left, and reports what it perceives to the station. When its residual battery is barely enough to reach its pad, it remembers its position, goes back to recharge and then resumes the exploration from where it left off.

In the first version, a scout moved randomly within its quadrant, with a preference for keeping its current direction to avoid continuous changes of course. This strategy did not guarantee coverage: scouts tended to revisit the same areas, while other parts of their quadrant remained unexplored for a long time. The current strategy gives the scouts a memory of the explored area: each scout keeps a counter for every cell, increased every time the cell is considered as a possible next step, and at every step it moves to the free neighboring cell with the lowest counter, avoiding the cell it just came from and breaking ties randomly. In this way the scout is naturally driven towards the least explored parts of its quadrant, while the randomness prevents different scouts from following the same patterns. Since this choice requires reading and updating the counters of all the candidate cells at every step, it was implemented as a Java internal action.

Firefighter drone

- **Beliefs**: its position, its battery and water tank levels, its home pad, its current status (idle, fighting a fire, refilling or recharging), the burning cells assigned by the station or perceived directly, its current target, the cells claimed by the nearby firefighters and which firefighters are currently within range.
- **Desires**: extinguishing every known fire, while preserving enough battery and water to keep operating.
- **Intentions**: when idle, the firefighter selects the nearest burning cell not claimed by others, claims it, reaches it and extinguishes it, then moves on to the nearest burning cell it perceives. When the tank is empty, it asks the station for the nearest known lake and goes there only if the lake is closer than the station and its battery

is enough to reach the lake and then the station; otherwise, it goes back to the station to both recharge and refill. Battery has the highest priority: as soon as it gets too low, the firefighter drops whatever it is doing, releases its target and goes back to its pad.

The resulting behavior can be summarized as a state machine over the status of the firefighter (Figure 4): since battery has the highest priority, the firefighter can go back to recharge from any state.

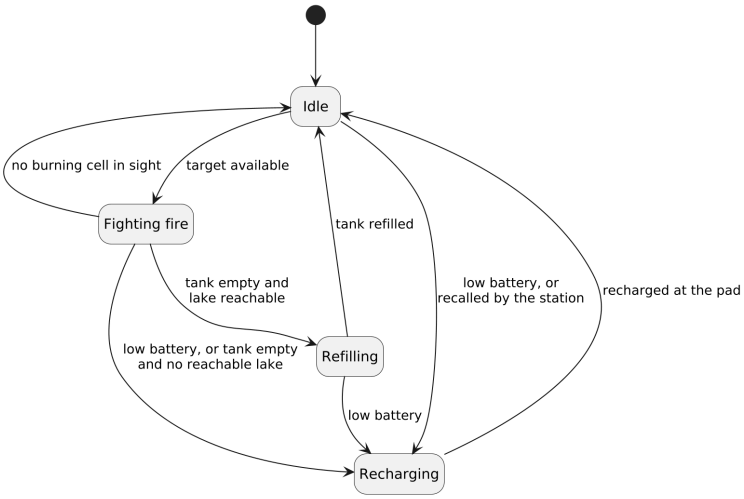


Figure 4: States of a firefighter drone

Since every message is handled by a separate intention, a firefighter receiving two recall messages almost at the same time could start two trips towards different destinations, oscillating in place and consuming battery twice as fast. To prevent this, the status check and update are performed in a short atomic step, while the trip itself runs as a separate intention, so that the drone remains responsive while flying.

Ranger station

- **Beliefs**: the global map of the forest built from the scouts' reports, the home pad of every drone, which firefighters are at the base or on their way back, and which burning cells have already been assigned.
- **Desires**: having every fire assigned to a firefighter and supporting firefighters in managing their resources.
- **Intentions**: the station is purely reactive. When a new burning cell appears in the map, it assigns it to the closest firefighter at the base, or to the others in turn if none is available; when a firefighter runs out of water it suggests the nearest known lake, and when a firefighter is idle away from the base it sends it back to its pad.

The dispatch policy of the station went through several versions. The first one assigned each burning cell to a firefighter chosen in round-robin among a number of firefighters proportional to the size of the fire; with small fires this number was one, so the same firefighter was always chosen, regardless of its distance from the fire. Policies based on fire zones, assigning a single firefighter to each group of nearby burning cells, and on a minimum reserve kept at the base were then evaluated, but they required the station to know where the firefighters were working, which meant either constant position updates or rough approximations. The final policy is the simplest one: the station dispatches the closest firefighter at the base for each newly detected burning cell, falling back to round-robin only when the base is empty, and leaves to the local coordination among firefighters the task of avoiding redundant interventions.

Battery management The battery parameters are an example of the initial beliefs computed from the configuration. The battery is expressed as a percentage, but the number of moves corresponding to one percentage point is computed from the map, so that a firefighter can always perform at least twice the distance between its pad and the farthest corner of the forest. Scouts, which do not carry a water tank, consume battery at half the rate of firefighters. In this way the autonomy of the drones scales with the size of the forest, while their behavior, which only reasons on percentages and distances, remains unchanged.

Battery management was initially delegated to the station, which performed all the distance computations and decided in place of the drones. Every drone sent its position and battery level at every step, and the station decided when to call it back; with eight drones, these updates were by far the most frequent messages, and when the station fell behind, recalls arrived too late and some drones ran out of battery. Since every drone already knows its own position, battery and home pad, the decision was moved to the drones and expressed as a logical rule, and the same was done for the choice between refilling at a lake or at the station. In this way the station is no longer involved in time-critical decisions, and drones return to recharge autonomously, as stated in the requirements.

3.4 Environment and Forest Model

The environment is the bridge between the agents and the model of the forest. It has four main tasks:

1. it keeps track of the position of every drone;
2. it provides the agents with their percepts;
3. it executes the actions requested by the agents;
4. it advances the time of the simulation, tick by tick.

The model, instead, only describes the forest and its evolution, and is never accessed directly by the agents.

3.4.1 Forest model

The forest is a grid of cells, each of which is in one of the following states: tree, empty (a clearing), burning, wet tree, water, station, charging pad of a scout, or charging pad of a firefighter. Drones are not part of the grid: since they fly, they do not change the state of the cells they pass over, and their positions are kept separately by the environment. For the same reason, trees and water are not obstacles for them.

The forest is generated procedurally at every run, in three steps:

1. the station is placed in the center of the grid, as a block of cells surrounded by the eight pads of the drones;
2. lakes grow from a few random seeds, expanding to the neighboring cells with a probability that decreases as they grow; isolated seeds are then removed, and the holes left inside the lakes are filled, so that their shores are regular;
3. trees are placed on random empty cells, leaving the remaining ones as clearings.

As a result, every run takes place in a different forest, and the agents cannot rely on any prior knowledge of the map.

3.4.2 Fire dynamics

Fires are handled entirely by the model, and agents do not reason about their propagation: they simply perceive the new burning cells. A new fire ignites on a random tree every 100 ticks, regardless of the fires that are already active; when the last active fire is extinguished, this countdown restarts, giving the forest a pause before the next ignition. Every 33 ticks, each burning tree may set fire to one of its neighboring trees, with a fixed probability. A tree left burning for 500 ticks is lost and becomes an empty cell, while an extinguished tree becomes wet and turns back into a regular tree after 200 ticks, when it can catch fire again. All these parameters can be changed in the configuration, to make the simulation more or less demanding for the drones.

3.4.3 Time model

The simulation evolves in discrete ticks, driven by the environment at a speed that the user can change from the graphical interface. Every action has a cost in ticks: moving costs one tick, while extinguishing, refilling and recharging last longer, and agents are suspended until the corresponding number of ticks has elapsed.

This was not the original model. Initially, every delay was measured in real time, but agents run asynchronously and independently from the environment, so there was no single notion of time shared by the whole system. As a consequence, the simulation could not be paused consistently: agents kept reasoning and moving, and the time elapsed during the pause was counted anyway when the simulation resumed, so that trees burned out and new fires ignited all at once. Making the environment the only owner of time solved the problem: stopping the clock now freezes fires and drones consistently, and the speed of the simulation can be changed without altering its logic.

3.4.4 Perception and actions

At every reasoning cycle, each drone receives from the environment the percepts listed in Table 1, computed on the current state of the world. The station receives no percepts at all: its knowledge of the forest comes entirely from the messages of the drones.

Percept	Received by	Meaning
`position(X, Y, S)`	drones	position of the drone and state of the cell below it
`cell(X, Y, S)`	drones	state of each cell within the vision radius
`border(X, Y)`	drones	cells of the vision radius outside the grid
`obstacle(X, Y)`	drones	neighboring cells that cannot be entered
`home(X, Y)`	drones	position of the drone's pad
`wait_time(T)`	drones	duration in ticks of the long operations
`bound(...)`	scouts	limits of the quadrant to explore
`neighbour(N)`	firefighters	firefighters within short range

Table 1: Percepts provided by the environment

Perception is partial: a drone only sees the cells within a circular radius of three cells around it. The same radius is used to model the short-range antenna: the `neighbour` percept tells a firefighter which other firefighters it can currently talk to, so the communication constraint is enforced through perception rather than by filtering messages. Obstacles are the cells outside the grid and the cells occupied by another drone, unless that drone is on its own pad, so that a drone can always land at home.

The actions that the drones can perform are listed in Table 2.

Action	Effect
`move(X, Y)`	moves the drone to an adjacent cell, then waits one tick
`wait(T)`	suspends the drone for `T` ticks
`fire_extinguished(X, Y)`	turns a burning cell into a wet tree

Table 2: Actions provided by the environment

3.5 Graphical Interface

3.5.1 View

The view renders the forest as a tiled map, in which every cell is drawn with a small sprite. All the sprites are loaded once at startup by a sprite repository, which reads them from the resources of the project and scales them to the size of a cell. To make the forest look less uniform, each cell is randomly assigned one of the available tree variants, which is kept for the whole simulation; every variant has a matching three-frame animation used when the tree is burning. The repository also provides the tiles for the shores of

the lakes, chosen according to the neighboring water cells, and the tiles composing the roof and the walls of the station.

Since most of the map never changes, rendering is split into two layers. The static part, made of grass, water, shores, station, pads and trees, is drawn once into an off-screen image when the simulation starts, and only the cells that change state are redrawn on it afterwards. The dynamic part is drawn on top of it at every repaint: the animated burning trees, a semi-transparent highlight of the cells currently perceived by the drones, which makes their vision radius visible, and finally the drones themselves. In this way, every repaint only costs a few drawing operations, regardless of the size of the map.

3.5.2 Controller

The controller connects the interface to the environment, and is the only component aware of real time. It owns a timer that, at every expiration, asks the environment to advance the simulation by one tick, updates the cells that changed state and repaints the view. The control panel above the map lets the user change the speed of the simulation, from 1 to 60 ticks per second, which only changes the period of the timer, and pause it. Pausing stops both the tick timer, so that fires and drones freeze consistently as described in the time model, and the animation of the flames. The pause button of the Jason console is supported as well: while it is active, the timer keeps running but does not advance the clock.

4 Deployment

The only prerequisites are `git` and a JDK 21. All other dependencies, including the Jason interpreter (v3.2.1), are downloaded automatically by Gradle through the included wrapper.

```
git clone https://github.com/sbrambalds/ForestGuard.git
cd ForestGuard
./gradlew runForestguardMas
```

The task launches the centralized Jason infrastructure on the `forestguard.mas2j` file. This opens both the Jason MAS console and the simulation window (Figure 5). Simulation parameters (grid size, fire frequency, spread probability, battery factor, ...) can be tuned in `model/Config.java`. The agents are declared in `forestguard.mas2j`; their names must match the ones in `model/Config.java`, which assigns each drone its home pad, so adding or renaming an agent requires updating both files.

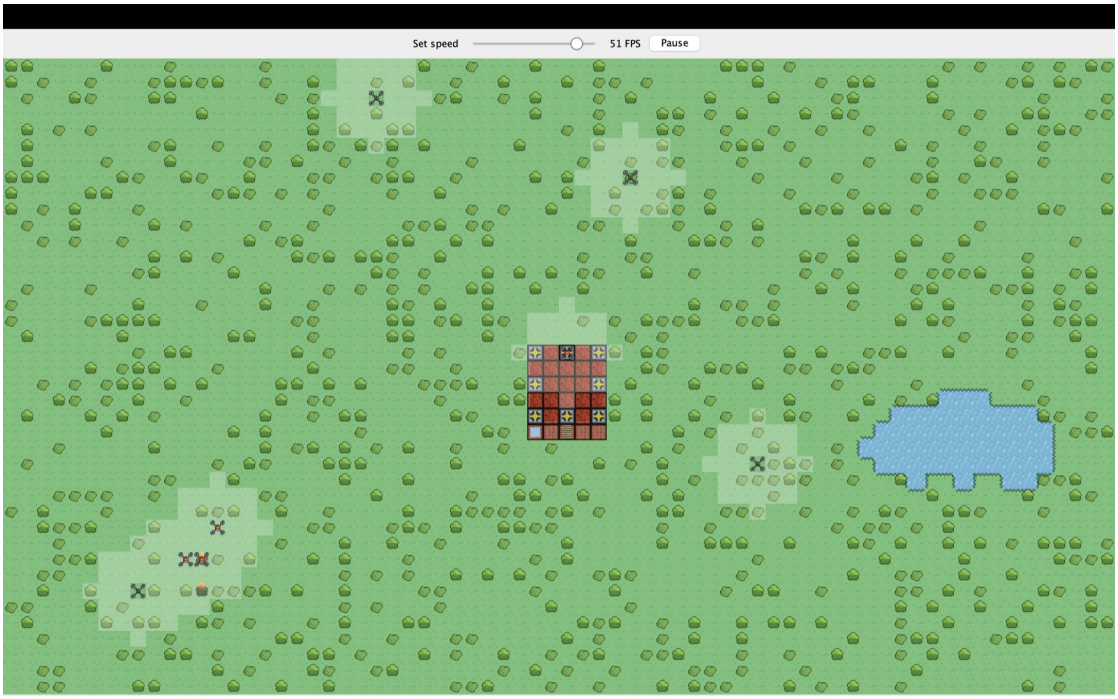


Figure 5: The simulation window: scouts and firefighters exploring the forest around the station, with the area perceived by the drones highlighted

5 Conclusions

ForestGuard shows how a swarm of autonomous drones, coordinated by a ranger station, can monitor a forest and fight fires without any human intervention. Scouts explore their own quadrant of the forest, driven towards the least visited areas, and keep the station's map up to date; the station dispatches firefighters to every newly detected fire; firefighters extinguish the fires, decide autonomously when to recharge or refill, and coordinate with the nearby ones to avoid targeting the same cell. All four goals of the project are therefore addressed: monitoring, fire suppression, resource management and coordination.

However, the current version still has some limitations, which suggest possible future developments:

- cells that burn out before being extinguished are never notified to the firefighters, so they may remain in their queues and be reached uselessly;
- drones move greedily towards their destination, one cell at a time; a path-planning algorithm, such as A*, would produce shorter and more reliable routes;
- the quadrants of the scouts are fixed: the station could use its global map to redirect them towards the least recently explored areas;

- the short-range communication constraint is respected by the agents, but not enforced by the infrastructure, which delivers any message regardless of the distance;
- the system has not been evaluated quantitatively: metrics such as the number of trees lost or the average time needed to extinguish a fire would allow different strategies and parameters to be compared.